

v2.0

Generated by Doxygen 1.17.0

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 Studentas Class Reference	7
4.1.1 Constructor & Destructor Documentation	8
4.1.1.1 Studentas() [1/4]	8
4.1.1.2 Studentas() [2/4]	8
4.1.1.3 ~Studentas()	8
4.1.1.4 Studentas() [3/4]	8
4.1.1.5 Studentas() [4/4]	9
4.1.2 Member Function Documentation	9
4.1.2.1 addPaz()	9
4.1.2.2 egzaminas() [1/2]	9
4.1.2.3 egzaminas() [2/2]	9
4.1.2.4 mediana() [1/2]	9
4.1.2.5 mediana() [2/2]	9
4.1.2.6 operator=() [1/2]	9
4.1.2.7 operator=() [2/2]	9
4.1.2.8 paz() [1/2]	9
4.1.2.9 paz() [2/2]	10
4.1.2.10 readStudent()	10
4.1.2.11 rezultatas() [1/2]	10
4.1.2.12 rezultatas() [2/2]	10
4.1.2.13 setEgzaminas()	10
4.1.2.14 setMediana()	10
4.1.2.15 setPaz() [1/2]	10
4.1.2.16 setPaz() [2/2]	10
4.1.2.17 setRezultatas()	10
4.1.2.18 skaiciuotiRezultata()	11
4.1.3 Friends And Related Symbol Documentation	11
4.1.3.1 operator<<	11
4.1.3.2 operator>>	11
4.2 Zmogus Class Reference	11
4.2.1 Constructor & Destructor Documentation	12
4.2.1.1 Zmogus() [1/2]	12
4.2.1.2 Zmogus() [2/2]	12

4.2.1.3 ~Zmogus()	12
4.2.2 Member Function Documentation	12
4.2.2.1 pavarde() [1/2]	12
4.2.2.2 pavarde() [2/2]	12
4.2.2.3 setPavarde()	12
4.2.2.4 setVardas()	12
4.2.2.5 skaiciuotiRezultata()	13
4.2.2.6 vardas() [1/2]	13
4.2.2.7 vardas() [2/2]	13
4.2.3 Member Data Documentation	13
4.2.3.1 pavarde_	13
4.2.3.2 vardas_	13
5 File Documentation	15
5.1 funkcions.h File Reference	15
5.1.1 Macro Definition Documentation	16
5.1.1.1 FUNKCIONS_H	16
5.1.2 Function Documentation	16
5.1.2.1 GeneruotiFaila()	16
5.1.2.2 Isvedimas()	16
5.1.2.3 IsvestilFaila()	16
5.1.2.4 LaikoMatavimas()	16
5.1.2.5 Nuskaitymas()	16
5.1.2.6 NuskaitymasIsFailo()	16
5.1.2.7 RandomPazymiai()	17
5.1.2.8 RandomStudentai()	17
5.1.2.9 RankinisIvedimas()	17
5.1.2.10 TestuotiGreiti()	17
5.2 funkcions.h	17
5.3 Generatorius.cpp File Reference	18
5.3.1 Function Documentation	18
5.3.1.1 GeneruotiFaila()	18
5.4 includes.h File Reference	18
5.4.1 Macro Definition Documentation	19
5.4.1.1 INCLUDES_H	19
5.5 includes.h	19
5.6 input_impl.h File Reference	20
5.6.1 Function Documentation	20
5.6.1.1 Nuskaitymas()	20
5.7 input_impl.h	20
5.8 inputbyhand_impl.h File Reference	21
5.8.1 Function Documentation	21

5.8.1.1 RankinisIvedimas()	21
5.9 inputbyhand_impl.h	21
5.10 LaikoMatavimas.cpp File Reference	23
5.10.1 Function Documentation	23
5.10.1.1 LaikoMatavimas()	23
5.11 nuskaitymas_impl.h File Reference	23
5.11.1 Function Documentation	23
5.11.1.1 NuskaitymasIsFailo()	23
5.12 nuskaitymas_impl.h	24
5.13 output_impl.h File Reference	24
5.13.1 Function Documentation	25
5.13.1.1 Isvedimas()	25
5.13.1.2 IsvestilFaila()	25
5.14 output_impl.h	25
5.15 pazymiairandom_impl.h File Reference	26
5.15.1 Function Documentation	26
5.15.1.1 RandomPazymiai()	26
5.16 pazymiairandom_impl.h	27
5.17 Strategijos.h File Reference	27
5.17.1 Macro Definition Documentation	28
5.17.1.1 STRATEGIJOS_H	28
5.17.2 Function Documentation	28
5.17.2.1 Strategija1()	28
5.17.2.2 Strategija2()	28
5.17.2.3 Strategija3()	28
5.18 Strategijos.h	29
5.19 studentairandom_impl.h File Reference	29
5.19.1 Function Documentation	29
5.19.1.1 RandomStudentai()	29
5.20 studentairandom_impl.h	30
5.21 Studentas.cpp File Reference	30
5.21.1 Function Documentation	31
5.21.1.1 comparePagalPavarde()	31
5.21.1.2 comparePagalRezultata()	31
5.21.1.3 comparePagalVarda()	31
5.21.1.4 operator<<()	31
5.21.1.5 operator>>()	31
5.22 Studentas.h File Reference	31
5.23 Studentas.h	32
5.24 testuoti_impl.h File Reference	32
5.24.1 Function Documentation	33
5.24.1.1 TestuotiGreiti()	33

5.25 testuoti_impl.h	33
5.26 v2.0.cpp File Reference	34
5.26.1 Typedef Documentation	34
5.26.1.1 StudContainer	34
5.26.2 Function Documentation	34
5.26.2.1 main()	34
5.27 Zmogus.h File Reference	34
5.28 Zmogus.h	35
Index	37

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Zmogus	11
Studentas	7

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas		7
Zmogus		11

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

funkcions.h	15
Generatorius.cpp	18
includes.h	18
input_impl.h	20
inputbyhand_impl.h	21
LaikoMatavimas.cpp	23
nuskaitymas_impl.h	23
output_impl.h	24
pazymiairandom_impl.h	26
Strategijos.h	27
studentairandom_impl.h	29
Studentas.cpp	30
Studentas.h	31
testuoti_impl.h	32
v2.0.cpp	34
Zmogus.h	34

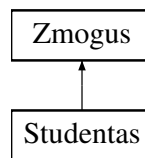
Chapter 4

Class Documentation

4.1 Studentas Class Reference

```
#include <Studentas.h>
```

Inheritance diagram for Studentas:



Public Member Functions

- `Studentas ()`
- `Studentas (std::istream &is)`
- `~Studentas ()=default`
- `Studentas (const Studentas &other)`
- `Studentas & operator= (const Studentas &other)`
- `Studentas (Studentas &&other) noexcept`
- `Studentas & operator= (Studentas &&other) noexcept`
- `void skaiciuotiRezultata () override`
- `std::istream & readStudent (std::istream &is)`
- `std::vector< int > & paz ()`
- `const std::vector< int > & paz () const`
- `int & egzaminas ()`
- `const int egzaminas () const`
- `double & rezultatas ()`
- `const double rezultatas () const`
- `double & mediana ()`
- `const double mediana () const`
- `void addPaz (int p)`
- `void setPaz (const std::vector< int > &p)`
- `void setPaz (std::vector< int > &&p)`
- `void setEgzaminas (int e)`
- `void setRezultatas (double r)`
- `void setMediana (double m)`

Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) ()=default
- [Zmogus](#) (const std::string &v, const std::string &p)
- virtual [~Zmogus](#) ()=default
- std::string & [vardas](#) ()
- const std::string & [vardas](#) () const
- std::string & [pavarde](#) ()
- const std::string & [pavarde](#) () const
- void [setVardas](#) (const std::string &v)
- void [setPavarde](#) (const std::string &p)

Friends

- std::istream & [operator>>](#) (std::istream &is, [Studentas](#) &s)
- std::ostream & [operator<<](#) (std::ostream &os, const [Studentas](#) &s)

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- std::string [vardas_](#)
- std::string [pavarde_](#)

4.1.1 Constructor & Destructor Documentation

4.1.1.1 [Studentas\(\)](#) [1/4]

```
Studentas::Studentas ()
```

4.1.1.2 [Studentas\(\)](#) [2/4]

```
Studentas::Studentas (
    std::istream & is)
```

4.1.1.3 [~Studentas\(\)](#)

```
Studentas::~~Studentas () [default]
```

4.1.1.4 [Studentas\(\)](#) [3/4]

```
Studentas::Studentas (
    const Studentas & other)
```

4.1.1.5 Studentas() [4/4]

```
Studentas::Studentas (  
    Studentas && other) [noexcept]
```

4.1.2 Member Function Documentation

4.1.2.1 addPaz()

```
void Studentas::addPaz (  
    int p) [inline]
```

4.1.2.2 egzaminas() [1/2]

```
int & Studentas::egzaminas () [inline]
```

4.1.2.3 egzaminas() [2/2]

```
const int Studentas::egzaminas () const [inline]
```

4.1.2.4 mediana() [1/2]

```
double & Studentas::mediana () [inline]
```

4.1.2.5 mediana() [2/2]

```
const double Studentas::mediana () const [inline]
```

4.1.2.6 operator=() [1/2]

```
Studentas & Studentas::operator= (  
    const Studentas & other)
```

4.1.2.7 operator=() [2/2]

```
Studentas & Studentas::operator= (  
    Studentas && other) [noexcept]
```

4.1.2.8 paz() [1/2]

```
std::vector< int > & Studentas::paz () [inline]
```

4.1.2.9 paz() [2/2]

```
const std::vector< int > & Studentas::paz () const [inline]
```

4.1.2.10 readStudent()

```
std::istream & Studentas::readStudent (
    std::istream & is)
```

4.1.2.11 rezultatas() [1/2]

```
double & Studentas::rezultatas () [inline]
```

4.1.2.12 rezultatas() [2/2]

```
const double Studentas::rezultatas () const [inline]
```

4.1.2.13 setEgzaminas()

```
void Studentas::setEgzaminas (
    int e) [inline]
```

4.1.2.14 setMediana()

```
void Studentas::setMediana (
    double m) [inline]
```

4.1.2.15 setPaz() [1/2]

```
void Studentas::setPaz (
    const std::vector< int > & p) [inline]
```

4.1.2.16 setPaz() [2/2]

```
void Studentas::setPaz (
    std::vector< int > && p) [inline]
```

4.1.2.17 setRezultatas()

```
void Studentas::setRezultatas (
    double r) [inline]
```

4.1.2.18 skaiciuotiRezultata()

```
void Studentas::skaiciuotiRezultata () [override], [virtual]
```

Implements [Zmogus](#).

4.1.3 Friends And Related Symbol Documentation

4.1.3.1 operator<<

```
std::ostream & operator<< (
    std::ostream & os,
    const Studentas & s) [friend]
```

4.1.3.2 operator>>

```
std::istream & operator>> (
    std::istream & is,
    Studentas & s) [friend]
```

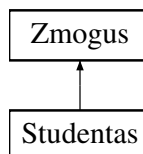
The documentation for this class was generated from the following files:

- [Studentas.h](#)
- [Studentas.cpp](#)

4.2 Zmogus Class Reference

```
#include <Zmogus.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- [Zmogus](#) ()=default
- [Zmogus](#) (const std::string &v, const std::string &p)
- virtual [~Zmogus](#) ()=default
- std::string & [vardas](#) ()
- const std::string & [vardas](#) () const
- std::string & [pavarde](#) ()
- const std::string & [pavarde](#) () const
- void [setVardas](#) (const std::string &v)
- void [setPavarde](#) (const std::string &p)
- virtual void [skaiciuotiRezultata](#) ()=0

Protected Attributes

- std::string [vardas_](#)
- std::string [pavarde_](#)

4.2.1 Constructor & Destructor Documentation

4.2.1.1 Zmogus() [1/2]

```
Zmogus::Zmogus () [default]
```

4.2.1.2 Zmogus() [2/2]

```
Zmogus::Zmogus (  
    const std::string & v,  
    const std::string & p) [inline]
```

4.2.1.3 ~Zmogus()

```
virtual Zmogus::~Zmogus () [virtual], [default]
```

4.2.2 Member Function Documentation

4.2.2.1 pavarde() [1/2]

```
std::string & Zmogus::pavarde () [inline]
```

4.2.2.2 pavarde() [2/2]

```
const std::string & Zmogus::pavarde () const [inline]
```

4.2.2.3 setPavarde()

```
void Zmogus::setPavarde (  
    const std::string & p) [inline]
```

4.2.2.4 setVardas()

```
void Zmogus::setVardas (  
    const std::string & v) [inline]
```

4.2.2.5 skaiciuotiRezultata()

```
virtual void Zmogus::skaiciuotiRezultata () [pure virtual]
```

Implemented in [Studentas](#).

4.2.2.6 vardas() [1/2]

```
std::string & Zmogus::vardas () [inline]
```

4.2.2.7 vardas() [2/2]

```
const std::string & Zmogus::vardas () const [inline]
```

4.2.3 Member Data Documentation

4.2.3.1 pavarde_

```
std::string Zmogus::pavarde_ [protected]
```

4.2.3.2 vardas_

```
std::string Zmogus::vardas_ [protected]
```

The documentation for this class was generated from the following file:

- [Zmogus.h](#)

Chapter 5

File Documentation

5.1 funkcions.h File Reference

```
#include "Studentas.h"
#include "includes.h"
#include "Strategijos.h"
#include "input_impl.h"
#include "inputbyhand_impl.h"
#include "pazymiairandom_impl.h"
#include "studentairandom_impl.h"
#include "nuskaitymas_impl.h"
#include "output_impl.h"
#include "testuoti_impl.h"
```

Macros

- `#define FUNKCIONS_H`

Functions

- `template<typename Container>`
`void RankinisIvedimas (Container &grupe)`
- `template<typename Container>`
`void RandomPazymiai (Container &grupe)`
- `template<typename Container>`
`void RandomStudentai (Container &grupe)`
- `template<typename Container>`
`void Nuskaitymas (Container &grupe, int &rusiavimas)`
- `template<typename Container>`
`void NuskaitymasIsFailo (const std::string &failas, Container &grupe)`
- `template<typename Container>`
`void Isvedimas (Container &grupe, int pasirinkimas, int rusiavimas)`
- `template<typename Container>`
`void IsvestiIfaila (const Container &grupe, const std::string &failas)`
- `template<typename Container>`
`void TestuotiGreiti (const string &failas)`
- `void GeneruotiFaila (const std::string &failas, int kiekis)`
- `void LaikoMatavimas ()`

5.1.1 Macro Definition Documentation

5.1.1.1 FUNKCIOMS_H

```
#define FUNKCIOMS_H
```

5.1.2 Function Documentation

5.1.2.1 GeneruotiFaila()

```
void GeneruotiFaila (  
    const std::string & failas,  
    int kiekis)
```

5.1.2.2 Isvedimas()

```
template<typename Container>  
void Isvedimas (  
    Container & grupe,  
    int pasirinkimas,  
    int rusiavimas)
```

5.1.2.3 IsvestiIFaila()

```
template<typename Container>  
void IsvestiIFaila (  
    const Container & grupe,  
    const std::string & failas)
```

5.1.2.4 LaikoMatavimas()

```
void LaikoMatavimas ()
```

5.1.2.5 Nuskaitymas()

```
template<typename Container>  
void Nuskaitymas (  
    Container & grupe,  
    int & rusiavimas)
```

5.1.2.6 NuskaitymasIsFailo()

```
template<typename Container>  
void NuskaitymasIsFailo (  
    const std::string & failas,  
    Container & grupe)
```

5.1.2.7 RandomPazymiai()

```
template<typename Container>
void RandomPazymiai (
    Container & grupe)
```

5.1.2.8 RandomStudentai()

```
template<typename Container>
void RandomStudentai (
    Container & grupe)
```

5.1.2.9 RankinisIvedimas()

```
template<typename Container>
void RankinisIvedimas (
    Container & grupe)
```

5.1.2.10 TestuotiGreiti()

```
template<typename Container>
void TestuotiGreiti (
    const string & failas)
```

5.2 funkcions.h

[Go to the documentation of this file.](#)

```
00001 #ifndef FUNKCIIONS_H //apsauga nuo keliu itraukimu
00002 #define FUNKCIIONS_H //apsauga nuo keliu itraukimu
00003
00004 #include "Studentas.h"
00005 #include "includes.h"
00006 #include "Strategijos.h"
00007
00008 template<typename Container>
00009 void RankinisIvedimas(Container& grupe);
00010
00011 template<typename Container>
00012 void RandomPazymiai(Container& grupe);
00013
00014 template<typename Container>
00015 void RandomStudentai(Container& grupe);
00016
00017 template<typename Container>
00018 void Nuskaitymas(Container& grupe, int &rusiavimas);
00019
00020 template<typename Container>
00021 void NuskaitymasIsFailo(const std::string& failas, Container& grupe);
00022
00023 template<typename Container>
00024 void Isvedimas(Container& grupe, int pasirinkimas, int rusiavimas);
00025
00026 template<typename Container>
00027 void IsvestiIFaila(const Container& grupe, const std::string& failas);
00028
00029 template<typename Container>
00030 void TestuotiGreiti(const string& failas);
00031
00032 void GeneruotiFaila(const std::string& failas, int kiekis);
00033 void LaikoMatavimas();
00034
```

```
00035 // Include template implementations
00036 #include "input_impl.h"
00037 #include "inputbyhand_impl.h"
00038 #include "pazymiairandom_impl.h"
00039 #include "studentairandom_impl.h"
00040 #include "nuskaitymas_impl.h"
00041 #include "output_impl.h"
00042 #include "testuoti_impl.h"
00043
00044 #endif //apsauga nuo keliu įtraukimų
```

5.3 Generatorius.cpp File Reference

```
#include "Studentas.h"
#include "funkcions.h"
#include "includes.h"
```

Functions

- void [GeneruotiFaila](#) (const string &failas, int kiekis)

5.3.1 Function Documentation

5.3.1.1 GeneruotiFaila()

```
void GeneruotiFaila (
    const string & failas,
    int kiekis)
```

5.4 includes.h File Reference

```
#include <iostream>
#include <string>
#include <vector>
#include <iomanip>
#include <algorithm>
#include <limits>
#include <cctype>
#include <cstdlib>
#include <ctime>
#include <fstream>
#include <chrono>
#include <sstream>
#include <stdexcept>
#include <random>
#include <list>
#include <deque>
#include <type_traits>
```

Macros

- `#define INCLUDES_H`

5.4.1 Macro Definition Documentation**5.4.1.1 INCLUDES_H**

```
#define INCLUDES_H
```

5.5 includes.h

[Go to the documentation of this file.](#)

```
00001 #ifndef INCLUDES_H //apsauga nuo kelių įtraukimų
00002 #define INCLUDES_H //apsauga nuo kelių įtraukimų
00003
00004 #include <iostream>
00005 #include <string>
00006 #include <vector>
00007 #include <iomanip>
00008 #include <algorithm>
00009 #include <limits>
00010 #include <cctype>
00011 #include <cstdlib>
00012 #include <ctime>
00013 #include <fstream>
00014 #include <chrono>
00015 #include <sstream>
00016 #include <stdexcept>
00017 #include <random>
00018 #include <list>
00019 #include <deque>
00020 #include <type_traits>
00021
00022 using std::string;
00023 using std::vector;
00024 using std::list;
00025 using std::deque;
00026 using std::cout;
00027 using std::endl;
00028 using std::cin;
00029 using std::left;
00030 using std::right;
00031 using std::setw;
00032 using std::fixed;
00033 using std::setprecision;
00034 using std::numeric_limits;
00035 using std::streamsize;
00036 using std::max;
00037 using std::ifstream;
00038 using std::getline;
00039 using std::ofstream;
00040 using std::mt19937;
00041 using std::uniform_int_distribution;
00042 using std::random_device;
00043 using std::chrono::high_resolution_clock;
00044 using std::chrono::duration;
00045 using std::to_string;
00046 using std::find; //naudojama funkcijoms, kurios ieško elementų konteineryje, naudojant lambda funkciją
00047 using std::find_if; //naudojama funkcijoms, kurios ieško elementų konteineryje su sąlyga, naudojant
    lambda funkciją
00048 using std::search; //naudojama funkcijoms, kurios ieško elementų konteineryje, naudojant lambda
    funkciją
00049 using std::copy; //naudojama funkcijoms, kurios kopijuoja elementus iš vieno konteinerio į kitą,
    naudojant lambda funkciją
00050 using std::remove; //naudojama funkcijoms, kurios pašalina elementus iš konteinerio, naudojant lambda
    funkciją
00051 using std::remove_if; //naudojama funkcijoms, kurios pašalina elementus iš konteinerio su sąlyga,
    naudojant lambda funkciją
00052 using std::remove_copy; //naudojama funkcijoms, kurios pašalina kopijas iš konteinerio, naudojant
    lambda funkciją
00053 using std::remove_copy_if; //naudojama funkcijoms, kurios pašalina kopijas iš konteinerio su sąlyga,
    naudojant lambda funkciją
00054 using std::transform; //naudojama funkcijai, kuri konvertuoja string į int, naudojant lambda funkciją
00055 using std::partition; //greitesnis nei stable_partition, bet nekeičia elementų tvarkos
00056 using std::stable_partition; //lėtesnis nei partition, bet nekeičia elementų tvarkos
00057
00058 #endif //apsauga nuo kelių įtraukimų
```

5.6 input_impl.h File Reference

```
#include "Studentas.h"
#include "includes.h"
```

Functions

- `template<typename Container>`
`void Nuskaitymas (Container &grupe, int &rusiavimas)`

5.6.1 Function Documentation

5.6.1.1 Nuskaitymas()

```
template<typename Container>
void Nuskaitymas (
    Container &grupe,
    int &rusiavimas)
```

5.7 input_impl.h

[Go to the documentation of this file.](#)

```
00001 #ifndef INPUT_IMPL_H
00002 #define INPUT_IMPL_H
00003
00004 #include "Studentas.h"
00005 #include "includes.h"
00006
00007 template<typename Container>
00008 void Nuskaitymas(Container& grupe, int &rusiavimas) {
00009
00010     Studentas A;
00011
00012     cout<<"Kaip norite rusiuoti duomenis? Pasirinkite: \n";
00013     cout<<"1 - rusiuoti pagal varda \n";
00014     cout<<"2 - rusiuoti pagal pavarde \n";
00015     cout<<"3 - rusiuoti pagal galutini rezultata \n";
00016     cin>>rusiavimas;
00017     std::ifstream fd("kursiokai.txt");
00018     string temp;
00019     getline(fd, temp); //nuskaitoma antraste
00020
00021     while(getline(fd, temp)) { //skaitomos visos kitos eilutės, kol jų yra
00022         std::stringstream ss(temp); //eilutė paverčiama į srauto objektą, kad būtų galima lengviau
00023         išskirti duomenis
00024         ss>>A.vardas()>>A.pavarde(); //iš srauto objekto išskiriami vardas ir pavardė
00025         vector<int> pazymiai; //laikina saugomi pazymiai
00026         int pazmys;
00027         int sum = 0;
00028         while(ss > pazmys) {
00029             pazymiai.push_back(pazmys); //pažymys pridedamas į laikinojo vektoriaus masyvą, kol yra skaitomų
00030             pažymių
00031         }
00032         A.egzaminas() = pazymiai.back(); //paima paskutinį pažymį kaip egzamino pažymį
00033         pazymiai.pop_back(); //pašalina paskutinį pažymį iš vektoriaus, nes jis jau yra priskirtas egzaminui
00034         A.paz() = pazymiai; //likusius pažymius priskiria namų darbų pažymiams
00035         for(int paz:A.paz()) {
00036             sum+=paz;
00037         }
00038         std::sort(A.paz().begin(), A.paz().end()); //surūšiuoja pažymius nuo mažiausio iki didžiausio
00039         int kiekis=A.paz().size();
```

```

00040     if(kiekis % 2 == 0)
00041 A.mediana() = (A.paz()[kiekis/2 - 1] + A.paz()[kiekis/2]) / 2.0;
00042     else
00043 A.mediana() = A.paz()[kiekis/2];
00044     // prevent division by zero
00045     if (!A.paz().empty())
00046 A.rezultatas()=(sum*1.0/A.paz().size())*0.4 + A.egzaminas()*0.6;
00047     else
00048 A.rezultatas() = 0;
00049     A.paz().clear(); //išvalomi pažymiai, kad būtų galima įvesti kitam studentui
00050     grupe.push_back(A); //studentas įdedamas į grupę
00051 }
00052 fd.close();
00053 }
00054
00055 #endif // INPUT_IMPL_H

```

5.8 inputbyhand_impl.h File Reference

```

#include "funkcions.h"
#include "Studentas.h"
#include "includes.h"

```

Functions

- `template<typename Container>`
`void RankinisIvedimas (Container &grupe)`

5.8.1 Function Documentation

5.8.1.1 RankinisIvedimas()

```

template<typename Container>
void RankinisIvedimas (
    Container & grupe)

```

5.9 inputbyhand_impl.h

[Go to the documentation of this file.](#)

```

00001 #ifndef INPUTBYHAND_IMPL_H
00002 #define INPUTBYHAND_IMPL_H
00003
00004 #include "funkcions.h"
00005 #include "Studentas.h"
00006 #include "includes.h"
00007
00008 template<typename Container>
00009 void RankinisIvedimas(Container& grupe){
00010     Studentas A;
00011     char testistud='t';
00012     while(testistud=='t' || testistud=='T') {
00013
00014         while(true) {
00015             cout<<"Iveskite varda ir pavarde: \n";
00016             try {
00017                 cin>>A.vardas()>>A.pavarde();
00018                 bool hasDigit = std::any_of(A.vardas().begin(), A.vardas().end(), [](unsigned char c){ return
std::isdigit(c); }) || std::any_of(A.pavarde().begin(), A.pavarde().end(), [](unsigned char c){ return
std::isdigit(c); });

```

```

00019 if(hasDigit) {
00020     throw std::runtime_error("Prasome ivesti tik raides!");
00021     continue; }
00022 break;
00023 }
00024     catch(const std::exception& e) {
00025         cout << e.what() << endl;
00026         cin.clear();
00027         cin.ignore(numeric_limits<streamsize>::max(), '\n');
00028     }
00029 }
00030
00031 char testipaz = 't';
00032 int temp; //laikinas pazymys
00033 int sum=0;
00034     cout<<"Iveskite visus pazymius\n";
00035
00036 while(testipaz=='t' || testipaz=='T') {
00037     while(true) {
00038         cout<<"iveskite pazymi: \n";
00039         try {
00040             cin >> temp;
00041             if(cin.fail()) {
00042                 throw std::runtime_error("Prasome ivesti tik skaicius!");
00043             }
00044             if(temp < 1 || temp > 10) {
00045                 throw std::out_of_range("Prasome ivesti skaiciu nuo 1 iki 10!");
00046             }
00047             A.paz().push_back(temp);
00048             sum += temp;
00049             break;
00050         }
00051         catch(const std::exception& e) {
00052             cout << e.what() << endl;
00053             cin.clear();
00054             cin.ignore(numeric_limits<streamsize>::max(), '\n');
00055         }
00056     }
00057
00058     cout<<"Spauskite t, jei norite ivesti dar viena pazymi "<<endl;
00059     cout<<"Jei baigete pazymiu ivedima, spauskite bet koki (ne t) simboli"<<endl;
00060     cin>>testipaz;
00061 }
00062
00063     while(true) {
00064         cout<<"iveskite egzamino pazymi: \n";
00065         try {
00066             cin>>A.egzaminas();
00067             if(cin.fail()) {
00068                 throw std::runtime_error("Prasome ivesti tik skaicius!");
00069             }
00070             if(A.egzaminas() < 1 || A.egzaminas() > 10) {
00071                 throw std::out_of_range("Prasome ivesti skaiciu nuo 1 iki 10!");
00072             }
00073             break;
00074         }
00075         catch(const std::exception& e) {
00076             cout << e.what() << endl;
00077             cin.clear();
00078             cin.ignore(numeric_limits<streamsize>::max(), '\n');
00079         }
00080     }
00081
00082     A.paz().push_back(A.egzaminas());
00083     std::sort(A.paz().begin(), A.paz().end());
00084
00085     int kiekis = A.paz().size();
00086     if(kiekis == 0) A.mediana() = 0;
00087     else if(kiekis % 2 == 0) A.mediana() = (A.paz()[kiekis/2 - 1] + A.paz()[kiekis/2]) / 2.0;
00088     else A.mediana() = A.paz()[kiekis/2];
00089     A.paz().pop_back();
00090
00091     A.rezultatas()=sum*1.0/(A.paz().size()*1.0)*0.4+A.egzaminas()*0.6;
00092     grupe.push_back(A);
00093     A.paz().clear();
00094     cout<<"Spauskite t, jei norite ivesti kita studenta "<<endl;
00095     cout<<"Jei baigete studentu ivedima, spauskite bet koki (ne t) simboli"<<endl;
00096     cin>>testistud;
00097 }
00098 }
00099
00100 #endif // INPUTBYHAND_IMPL_H

```

5.10 LaikoMatavimas.cpp File Reference

```
#include "Studentas.h"  
#include "funkcions.h"  
#include "includes.h"
```

Functions

- void [LaikoMatavimas](#) ()

5.10.1 Function Documentation

5.10.1.1 LaikoMatavimas()

```
void LaikoMatavimas ()
```

5.11 nuskaitymas_impl.h File Reference

```
#include "Studentas.h"  
#include "includes.h"
```

Functions

- template<typename Container>
void [NuskaitymasIsFailo](#) (const std::string &failas, Container &grupe)

5.11.1 Function Documentation

5.11.1.1 NuskaitymasIsFailo()

```
template<typename Container>  
void NuskaitymasIsFailo (  
    const std::string & failas,  
    Container & grupe)
```

5.12 nuskaitymas_impl.h

[Go to the documentation of this file.](#)

```

00001 #ifndef NUSKAITYMAS_IMPL_H
00002 #define NUSKAITYMAS_IMPL_H
00003
00004 #include "Studentas.h"
00005 #include "includes.h"
00006
00007 template<typename Container>
00008 void NuskaitymasIsFailo(const std::string& failas, Container& grupe) {
00009
00010     ifstream fd(failas);
00011     if(!fd) {
00012         cout<<"FAILAS NERASTAS: "<<failas<<endl;
00013         return; }
00014
00015     string temp;
00016     getline(fd, temp);
00017
00018     while(getline(fd, temp)) {
00019         Studentas A;
00020         std::stringstream ss(temp);
00021         string vardas,pavarde;
00022         ss>>vardas>>pavarde;
00023
00024         A.setVardas(vardas);
00025         A.setPavarde(pavarde);
00026         vector<int> pazymiai;
00027         int pazymys;
00028         int sum = 0;
00029
00030         while(ss>>pazymys)
00031             pazymiai.push_back(pazymys);
00032         A.setEgzaminas(pazymiai.back());
00033         pazymiai.pop_back();
00034         A.setPaz(pazymiai);
00035         for(int p : A.paz())
00036             sum += p;
00037
00038         vector<int> tempPaz = A.paz();
00039         std::sort(tempPaz.begin(), tempPaz.end());
00040         int kiekis = tempPaz.size();
00041         double mediana;
00042         if(kiekis % 2 == 0)
00043             mediana = (tempPaz[kiekis/2-1] + tempPaz[kiekis/2]) / 2.0;
00044         else
00045             mediana = tempPaz[kiekis/2];
00046
00047         A.setMediana(mediana);
00048
00049         double rez = (sum * 1.0 / A.paz().size()) * 0.4 + A.egzaminas() * 0.6;
00050         A.setRezultatas(rez);
00051         grupe.push_back(A);
00052     }
00053     fd.close();
00054 }
00055
00056 #endif // NUSKAITYMAS_IMPL_H

```

5.13 output_impl.h File Reference

```

#include "Studentas.h"
#include "includes.h"

```

Functions

- template<typename Container>
void [IsvestiIfFaila](#) (const Container &grupe, const std::string &failas)
- template<typename Container>
void [Isvedimas](#) (Container &grupe, int pasirinkimas, int rusiavimas)

5.13.1 Function Documentation

5.13.1.1 Isvedimas()

```
template<typename Container>
void Isvedimas (
    Container & grupe,
    int pasirinkimas,
    int rusiavimas)
```

5.13.1.2 IsvestiIFaila()

```
template<typename Container>
void IsvestiIFaila (
    const Container & grupe,
    const std::string & failas)
```

5.14 output_impl.h

[Go to the documentation of this file.](#)

```
00001 #ifndef OUTPUT_IMPL_H
00002 #define OUTPUT_IMPL_H
00003
00004 #include "Studentas.h"
00005 #include "includes.h"
00006
00007 template<typename Container>
00008 void IsvestiIFaila(const Container& grupe, const std::string& failas) {
00009     ofstream fr(failas);
00010     fr<<left<setw(20)<<"Vardas"<<setw(20)<<"Pavarde"<<setw(20)<<"Galutinis\n";
00011     for(const auto& A : grupe) {
00012         fr<<left<setw(20)<<A.vardas()<<setw(20)<<A.pavarde()<<setw(20)<<fixed<setprecision(2)<<A.rezultatas()<<"\n";
00013     }
00014     fr.close();
00015 }
00016
00017 template<typename Container>
00018 void Isvedimas(Container& grupe, int pasirinkimas, int rusiavimas) {
00019     Container vargsiukai;
00020     Container kietiakai;
00021
00022     auto pagalVarda = [](const Studentas &a, const Studentas &b) {
00023         return a.vardas() < b.vardas(); };
00024
00025     auto pagalPavarde = [](const Studentas &a, const Studentas &b) {
00026         return a.pavarde() < b.pavarde(); };
00027
00028     auto pagalRez = [](const Studentas &a, const Studentas &b) {
00029         return a.rezultatas() < b.rezultatas(); };
00030
00031     if(pasirinkimas == 4) {
00032         if constexpr (std::is_same<Container, std::list<Studentas>>::value) {
00033             if(rusiavimas==1) grupe.sort(pagalVarda);
00034             else if(rusiavimas==2) grupe.sort(pagalPavarde);
00035             else if(rusiavimas==3) grupe.sort(pagalRez);
00036             else cout<<"Prasome pasirinkti tik 1, 2 arba 3!\n"; }
00037         else {
00038             if(rusiavimas==1)
00039                 std::sort(grupe.begin(), grupe.end(), pagalVarda);
00040             else if(rusiavimas==2)
00041                 std::sort(grupe.begin(), grupe.end(), pagalPavarde);
00042             else if(rusiavimas==3)
00043                 std::sort(grupe.begin(), grupe.end(), pagalRez);
00044             else cout<<"Prasome pasirinkti tik 1, 2 arba 3!\n"; }
00045         }
00046     }
00047     else { cout<<"Prasome pasirinkti tik 1, 2 arba 3!\n"; }
00048 }
```

```

00049 for(const auto& stud : grupe) {
00050     if(stud.rezultatas() < 5.0)
00051         vargsiukai.push_back(stud);
00052     else
00053         kietiakai.push_back(stud); }
00054
00055 IvestiIFaila(vargsiukai,"vargsiukai.txt");
00056 IvestiIFaila(kietiakai,"kietiakai.txt");
00057
00058 int isvedimas;
00059 while(true) {
00060     cout<<"Kur norite matyti rezultatus? \n";
00061     cout<<"1 - ekrane \n";
00062     cout<<"2 - faile rezultatai.txt \n";
00063     cin>>isvedimas;
00064     if(cin.fail() || (isvedimas!=1 && isvedimas!=2)) {
00065         cout<<"Prasome pasirinkti tik skaiciu 1 arba 2! \n";
00066         cin.clear();
00067         cin.ignore(numeric_limits<streamsize>::max(), '\n');
00068     }
00069     else break;
00070 }
00071
00072 auto start = std::chrono::high_resolution_clock::now();
00073 if(isvedimas==1) {
00074     cout<<left<<setw(20)<<"Vardas"<<setw(20)<<"Pavarde"<<setw(20)<<"Galutinis"<<"\n";
00075     for(const auto &A : grupe) {
00076         cout<<left<<setw(20)<<A.vardas()<<setw(20)<<A.pavarde()<<setw(20)<<fixed<<setprecision(2)<<
00077         A.rezultatas()<<"\n";
00078     }
00079     else if(isvedimas==2){
00080 ofstream fr("rezultatai.txt");
00081         fr<<left<<setw(20)<<"Vardas"<<setw(20)<<"Pavarde"<<setw(20)<<"Galutinis"<<"\n";
00082         for(const auto &A : grupe) {
00083             fr<<left<<setw(20)<<A.vardas()<<setw(20)<<A.pavarde()<<setw(20)<<fixed<<setprecision(2)<<A.rezultatas()<<"\n";
00084         }
00085         fr.close();
00086         cout<<"Prasome patikrinti faila rezultatai.txt \n";
00087     }
00088     auto end = std::chrono::high_resolution_clock::now();
00089     std::chrono::duration<double> diff = end - start;
00090     cout<<"Duomeniu isvedimo laikas: "<<diff.count()<<" s \n";
00091 }
00092
00093
00094 #endif // OUTPUT_IMPL_H

```

5.15 pazymiairandom_impl.h File Reference

```

#include "Studentas.h"
#include "includes.h"

```

Functions

- `template<typename Container>`
`void RandomPazymiai (Container &grupe)`

5.15.1 Function Documentation

5.15.1.1 RandomPazymiai()

```

template<typename Container>
void RandomPazymiai (
    Container & grupe)

```

5.16 pazymiairandom_impl.h

[Go to the documentation of this file.](#)

```

00001 #ifndef PAZYMIARANDOM_IMPL_H
00002 #define PAZYMIARANDOM_IMPL_H
00003
00004 #include "Studentas.h"
00005 #include "includes.h"
00006
00007 template<typename Container>
00008 void RandomPazymiai(Container& grupe) {
00009
00010     Studentas A;
00011
00012     char testistud='t';
00013     while(testistud=='t' || testistud=='T') {
00014         int sum=0;
00015         while(true) {
00016             cout<<"Iveskite varda ir pavarde: \n";
00017             try {
00018                 cin>>A.vardas()>>A.pavarde();
00019                 bool hasDigit = std::any_of(A.vardas().begin(), A.vardas().end(), [](unsigned char c){ return
std::isdigit(c); }) || std::any_of(A.pavarde().begin(), A.pavarde().end(), [](unsigned char c){ return
std::isdigit(c); });
00020                 if(hasDigit) {
00021                     throw std::runtime_error("Prasome ivesti tik raides!");
00022                     continue;
00023                 }
00024             }
00025             catch(const std::exception& e) {
00026                 cout << e.what() << endl;
00027                 cin.clear();
00028                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
00029             }
00030         }
00031
00032         char testipaz='t';
00033         int temp;
00034
00035         cout<<"Random namu darbu pazymiai:\n";
00036         while(testipaz=='t' || testipaz=='T') {
00037             temp = (rand() % 10) + 1;
00038             cout<<"Random pazymys: " << temp << endl;
00039
00040             A.paz().push_back(temp);
00041             sum+=temp;
00042             cout<<"Spauskite t, jei norite ivesti kita pazymi " << endl;
00043             cout<<"Jei baigete pazymiu ivedima, spauskite bet koki (ne t) simboli" << endl;
00044             cin>>testipaz;
00045         }
00046
00047         A.egzaminas() = (rand() % 10) + 1;
00048         cout<<"Sugeneruotas egzamino pazymys: " << A.egzaminas() << endl;
00049
00050         A.paz().push_back(A.egzaminas());
00051
00052         std::sort(A.paz().begin(), A.paz().end());
00053
00054         int kiekis = A.paz().size();
00055         if(kiekis == 0) A.mediana() = 0;
00056         else if(kiekis % 2 == 0) A.mediana() = (A.paz()[kiekis/2 - 1] + A.paz()[kiekis/2]) / 2.0;
00057         else A.mediana() = A.paz()[kiekis/2];
00058
00059         A.paz().pop_back();
00060
00061         A.rezultatas() = (sum * 1.0 / A.paz().size()) * 0.4 + A.egzaminas() * 0.6;
00062         grupe.push_back(A);
00063         A.paz().clear();
00064         cout<<"Spauskite t, jei norite ivesti kita studenta " << endl;
00065         cout<<"Jei baigete studentu ivedima, spauskite bet koki (ne t) simboli" << endl;
00066         cin>>testistud;
00067     }
00068 }
00069
00070 #endif // PAZYMIARANDOM_IMPL_H

```

5.17 Strategijos.h File Reference

```

#include "Studentas.h"
#include "funkcions.h"

```

```
#include "includes.h"
```

Macros

- #define [STRATEGIJOS_H](#)

Functions

- template<typename Container>
void [Strategija1](#) (Container &studentai, Container &vargsiukai, Container &kietiakai)
- template<typename Container>
void [Strategija2](#) (Container &studentai, Container &vargsiukai)
- template<typename Container>
void [Strategija3](#) (Container &studentai, Container &vargsiukai, Container &kietiakai)

5.17.1 Macro Definition Documentation

5.17.1.1 STRATEGIJOS_H

```
#define STRATEGIJOS_H
```

5.17.2 Function Documentation

5.17.2.1 Strategija1()

```
template<typename Container>  
void Strategija1 (  
    Container & studentai,  
    Container & vargsiukai,  
    Container & kietiakai)
```

5.17.2.2 Strategija2()

```
template<typename Container>  
void Strategija2 (  
    Container & studentai,  
    Container & vargsiukai)
```

5.17.2.3 Strategija3()

```
template<typename Container>  
void Strategija3 (  
    Container & studentai,  
    Container & vargsiukai,  
    Container & kietiakai)
```

5.18 Strategijos.h

[Go to the documentation of this file.](#)

```
00001 #ifndef STRATEGIJOS_H //apsauga nuo kelių įtraukimų
00002 #define STRATEGIJOS_H //apsauga nuo kelių įtraukimų
00003
00004 #include "Studentas.h"
00005 #include "funkcijos.h"
00006 #include "includes.h"
00007
00008 template<typename Container>
00009 void Strategija1(Container& studentai, Container& vargsiukai, Container& kietiakai) {
00010     for(const auto& s : studentai) {
00011         if(s.rezultatas() < 5)
00012             vargsiukai.push_back(s);
00013         else
00014             kietiakai.push_back(s); }
00015     }
00016
00017 template<typename Container>
00018 void Strategija2(Container& studentai, Container& vargsiukai) {
00019     auto it = std::remove_if(studentai.begin(), studentai.end(), [&](const Studentas& s){
00020         if(s.rezultatas() < 5) {
00021             vargsiukai.push_back(s);
00022         }
00023         return true;
00024     });
00025     studentai.erase(it, studentai.end());
00026 }
00027
00028 template<typename Container>
00029 void Strategija3(Container& studentai, Container& vargsiukai, Container& kietiakai) {
00030     auto it = std::partition (studentai.begin(), studentai.end(), [&](const Studentas& s) { //padalina
00031         studentus į dvi grupes
00032         return s.rezultatas() < 5; }); //grupė su vargšiuokais bus prieš grupę su kietiakais
00033     return s.rezultatas() < 5; });
00034     vargsiukai.insert(vargsiukai.end(), studentai.begin(), it); //vargsiukai konteineris užpildomas
00035     vargšiuokais
00036     kietiakai.insert(kietiakai.end(), it, studentai.end()); //kietiakai konteineris užpildomas kietiakais
00037 }
00038 #endif //apsauga nuo kelių įtraukimų
```

5.19 studentairandom_impl.h File Reference

```
#include "Studentas.h"
#include "includes.h"
```

Functions

- template<typename Container>
void RandomStudentai (Container &grupe)

5.19.1 Function Documentation

5.19.1.1 RandomStudentai()

```
template<typename Container>
void RandomStudentai (
    Container & grupe)
```

5.20 studentairrandom_impl.h

[Go to the documentation of this file.](#)

```

00001 #ifndef STUDENTAIRRANDOM_IMPL_H
00002 #define STUDENTAIRRANDOM_IMPL_H
00003
00004 #include "Studentas.h"
00005 #include "includes.h"
00006
00007 template<typename Container>
00008 void RandomStudentai(Container& grupe) {
00009
00010     Studentas A;
00011
00012     vector<string>
00013     firstNames={"Jonas","Petras","Ona","Lina","Mantas","Egle","Darius","Ruta","Tomas","Ieva"};
00014     vector<string>
00015     lastNames={"Petraitis","Kazlauskas","Jankauskas","Stankevicius","Vilkas","Daugela","Miskinis","Jankauskiene","Zalys","B"};
00016
00017     char testi='t';
00018     int ii=1;
00019
00020     while(testi=='t' || testi=='T') {
00021         A.vardas() = firstNames[rand() % firstNames.size()];
00022         A.pavarde() = lastNames[rand() % lastNames.size()];
00023
00024         int n = rand()%10 + 1;
00025         int sum = 0;
00026
00027         cout<<"Sugeneruotas " <<ii<<" studentas: " <<A.vardas() <<" " <<A.pavarde() <<"\n";
00028         cout<<"Random namu darbu pazymiai:\n";
00029
00030         for(int i=0;i<n;i++) {
00031             int temp = rand()%10 + 1;
00032             A.paz().push_back(temp);
00033             sum += temp;
00034             cout<<temp<<" ";
00035         }
00036         cout<<endl;
00037
00038         A.egzaminas() = (rand() % 10) + 1;
00039         cout<<"Random egzamino pazymys: " <<A.egzaminas() <<endl;
00040
00041         A.paz().push_back(A.egzaminas());
00042
00043         std::sort(A.paz().begin(), A.paz().end());
00044
00045         int kiekis = A.paz().size();
00046         if(kiekis == 0) A.mediana() = 0;
00047         else if(kiekis % 2 == 0) A.mediana() = (A.paz()[kiekis/2 - 1] + A.paz()[kiekis/2]) / 2.0;
00048         else A.mediana() = A.paz()[kiekis/2];
00049
00050         A.paz().pop_back();
00051
00052         A.rezultatas() = sum*1.0/(n*1.0)*0.4 + A.egzaminas()*0.6;
00053         grupe.push_back(A);
00054         A.paz().clear();
00055         cout<<"Sauskite t, jei norite sugeneruoti kita studenta " <<endl;
00056         cout<<"Jei baigete studentu generavima, spauskite bet koki (ne t) simboli" <<endl;
00057         cin>>testi;
00058         ii++;
00059     }
00060 }
00061 #endif // STUDENTAIRRANDOM_IMPL_H

```

5.21 Studentas.cpp File Reference

```

#include "Studentas.h"
#include "Zmogus.h"
#include <algorithm>
#include <numeric>
#include <utility>

```

Functions

- `std::istream & operator>>` (`std::istream &is`, `Studentas &s`)
- `std::ostream & operator<<` (`std::ostream &os`, `const Studentas &s`)
- `bool comparePagalVarda` (`const Studentas &a`, `const Studentas &b`)
- `bool comparePagalPavarde` (`const Studentas &a`, `const Studentas &b`)
- `bool comparePagalRezultata` (`const Studentas &a`, `const Studentas &b`)

5.21.1 Function Documentation

5.21.1.1 `comparePagalPavarde()`

```
bool comparePagalPavarde (  
    const Studentas & a,  
    const Studentas & b)
```

5.21.1.2 `comparePagalRezultata()`

```
bool comparePagalRezultata (  
    const Studentas & a,  
    const Studentas & b)
```

5.21.1.3 `comparePagalVarda()`

```
bool comparePagalVarda (  
    const Studentas & a,  
    const Studentas & b)
```

5.21.1.4 `operator<<()`

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Studentas & s)
```

5.21.1.5 `operator>>()`

```
std::istream & operator>> (  
    std::istream & is,  
    Studentas & s)
```

5.22 Studentas.h File Reference

```
#include "Zmogus.h"
```

Classes

- class [Studentas](#)

5.23 Studentas.h

[Go to the documentation of this file.](#)

```
00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include "Zmogus.h"
00005
00006 class Studentas : public Zmogus { // Studentas paveldi iš Žmogus, todėl jis turi visus Žmogaus
    duomenis ir funkcijas
00007
00008 private:
00009
00010     std::vector<int> paz_;
00011     int egzaminas_;
00012     double rezultatas_;
00013     double mediana_;
00014
00015 public:
00016
00017     //Konsgtruktoriai ir destruktorius
00018     Studentas(); // numatytasis konstruktorius, kuris leidžia sukurti tuščią studentą, jei reikia
00019     Studentas(std::istream& is); // konstruktorius, kuris leidžia sukurti studentą iš įvesties srauto
00020     ~Studentas() = default; // destruktorius, kuris leidžia saugiai sunaikinti studento objektą, kai jis
    nebereikalingas
00021
00022     // Rule of Five
00023     Studentas(const Studentas& other);
00024     Studentas& operator=(const Studentas& other);
00025     Studentas(Studentas&& other) noexcept;
00026     Studentas& operator=(Studentas&& other) noexcept;
00027
00028     // Override iš Zmogus
00029     void skaiciuotiRezultata() override; //override nurodo, kad ši funkcija yra perrašymas iš bazinės
    klasės Zmogus, ir kad ji turi būti įgyvendinta Studentas klasėje
00030
00031     std::istream& readStudent(std::istream& is); // funkcija, kuri leidžia nuskaityti studento duomenis iš
    įvesties srauto, ir grąžina tą patį srautą, kad būtų galima grandinti įvestis
00032
00033     friend std::istream& operator>(std::istream& is, Studentas& s); //friend funkcija laidžia pasiekti
    Studentas klasės privačius duomenis
00034     friend std::ostream& operator<(std::ostream& os, const Studentas& s);
00035
00036     // Getteriai
00037     std::vector<int>& paz(){ return paz_; }
00038     const std::vector<int>& paz() const { return paz_; }
00039
00040     int& egzaminas() { return egzaminas_; }
00041     const int egzaminas() const { return egzaminas_; }
00042     double& rezultatas() { return rezultatas_; }
00043     const double rezultatas() const { return rezultatas_; }
00044     double& mediana() { return mediana_; }
00045     const double mediana() const { return mediana_; }
00046
00047     // Setteriai
00048     void addPaz(int p){ paz_.push_back(p); }
00049     void setPaz(const std::vector<int>& p){ paz_ = p; }
00050     void setPaz(std::vector<int>&& p){ paz_ = std::move(p); }
00051     void setEgzaminas(int e){ egzaminas_ = e; }
00052     void setRezultatas(double r){ rezultatas_ = r; }
00053     void setMediana(double m){ mediana_ = m; }
00054 };
00055
00056 #endif
```

5.24 testuoti_impl.h File Reference

```
#include "Studentas.h"
#include "funkcions.h"
#include "includes.h"
#include "Strategijos.h"
```

Functions

- `template<typename Container>`
`void TestuotiGreiti (const string &failas)`

5.24.1 Function Documentation

5.24.1.1 TestuotiGreiti()

```
template<typename Container>
void TestuotiGreiti (
    const string & failas)
```

5.25 testuoti_impl.h

[Go to the documentation of this file.](#)

```
00001 #ifndef TESTUOTI_IMPL_H
00002 #define TESTUOTI_IMPL_H
00003
00004 #include "Studentas.h"
00005 #include "funkcions.h"
00006 #include "includes.h"
00007 #include "Strategijos.h"
00008
00009 template<typename Container>
00010 void TestuotiGreiti(const string& failas) {
00011
00012     Container studentai;
00013     auto pr = std::chrono::high_resolution_clock::now();
00014     NuskaitymasIsFailo(failas, studentai);
00015     auto pa = std::chrono::high_resolution_clock::now();
00016
00017     cout<<failas<<"\n";
00018     cout<<"Nuskaitymas: "<<duration<double>(pa-pr).count()<<" s\n";
00019
00020     Container rusiuoti = studentai;
00021     auto prad = std::chrono::high_resolution_clock::now();
00022     if constexpr (std::is_same_v<Container, list<Studentas>>) {
00023         rusiuoti.sort([](const Studentas& a, const Studentas& b) {
00024             return a.rezultatas() < b.rezultatas(); });
00025     }
00026     else {
00027         sort(rsiuoti.begin(), rsiuoti.end(), [](const Studentas& a, const Studentas& b) {
00028             return a.rezultatas() < b.rezultatas(); });
00029     }
00030     auto pab = std::chrono::high_resolution_clock::now();
00031     cout<<"Rusiavimas: "<<duration<double>(pab-prad).count()<<" s\n\n";
00032
00033     // 1 Strategija
00034     {
00035         Container kopija = rusiuoti;
00036         Container vargsiukai, kietiakai;
00037         auto b = std::chrono::high_resolution_clock::now();
00038         Strategija1(kopija, vargsiukai, kietiakai);
00039         auto e = std::chrono::high_resolution_clock::now();
00040         cout<<"Strategija 1: "<<duration<double>(e-b).count()<<" s\n";
00041         IvestiIFaila(vargsiukai, "vargsiukai_" + failas);
00042         IvestiIFaila(kietiakai, "kietiakai_" + failas);
00043     }
00044
00045     // 2 Strategija
00046     {
00047         Container kopija = rusiuoti;
00048         Container vargsiukai;
00049         auto b = std::chrono::high_resolution_clock::now();
00050         Strategija2(kopija, vargsiukai);
00051         auto e = std::chrono::high_resolution_clock::now();
00052         cout<<"Strategija 2: "<<duration<double>(e-b).count()<<" s\n";
00053         IvestiIFaila(vargsiukai, "vargsiukai_" + failas);
00054         IvestiIFaila(kopija, "kietiakai_" + failas);
```

```

00055 }
00056
00057 // 3 Strategija
00058 {
00059     Container kopija = rusiuoti;
00060     Container vargsiukai, kietiakai;
00061     auto b = std::chrono::high_resolution_clock::now();
00062     Strategija3(kopija, vargsiukai, kietiakai);
00063     auto e = std::chrono::high_resolution_clock::now();
00064     cout<<"Strategija 3: "«duration<double>(e-b).count()«" s\n\n";
00065     IvestiIFaila(vargsiukai, "vargsiukai_" + failas);
00066     IvestiIFaila(kietiakai, "kietiakai_" + failas);
00067 }
00068 }
00069
00070 #endif // TESTUOTI_IMPL_H

```

5.26 v2.0.cpp File Reference

```

#include "Studentas.h"
#include "includes.h"
#include "funkcijos.h"
#include "Strategijos.h"

```

Typedefs

- using [StudContainer](#) = std::vector<[Studentas](#)>

Functions

- int [main](#) ()

5.26.1 Typedef Documentation

5.26.1.1 StudContainer

```
using StudContainer = std::vector<Studentas>
```

5.26.2 Function Documentation

5.26.2.1 main()

```
int main ()
```

5.27 Zmogus.h File Reference

```
#include "includes.h"
```

Classes

- class [Zmogus](#)

5.28 Zmogus.h

[Go to the documentation of this file.](#)

```
00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include "includes.h"
00005
00006 class Zmogus { // abstrakti bazinė klasė, objektų joje negalime kurti
00007
00008 protected: // Žmogaus duomenys (vardas, pavardė) turi būti pasiekiami tik Studentas klasei, todėl
00009     protected
00010     std::string vardas_;
00011     std::string pavarde_;
00012
00013 public: //Konstruktoriai ir destruktorius
00014     Zmogus() = default; // numatytasis konstruktorius, kuris leidžia sukurti tuščią žmogų, jei reikia
00015     Zmogus(const std::string& v, const std::string& p) : vardas_(v), pavarde_(p) {} // konstruktorius,
00016     kuris leidžia sukurti žmogų su vardu ir pavarde
00017
00018     virtual ~Zmogus() = default; //objekto naukinimas per virtualų destruktorių
00019
00020 // Getteriai
00021     std::string& vardas() { return vardas_; } // leidžia keisti vardą
00022     const std::string& vardas() const { return vardas_; } // leidžia gauti vardą, bet neleidžia jo keisti
00023     (const)
00024     std::string& pavarde() { return pavarde_; } // leidžia keisti pavardę
00025     const std::string& pavarde() const { return pavarde_; } // leidžia gauti pavardę, bet neleidžia jos
00026     keisti (const)
00027
00028     void setVardas(const std::string& v){ vardas_ = v; }
00029     void setPavarde(const std::string& p){ pavarde_ = p; }
00030
00031     virtual void skaiciuotiRezultata() = 0; //Klasė Žmogus tampa abstrakti, pačioje joje negalima kurti
00032     objekto
00033 };
00034 #endif
```


Index

- ~Studentas
 - Studentas, 8
- ~Zmogus
 - Zmogus, 12
- addPaz
 - Studentas, 9
- comparePagalPavarde
 - Studentas.cpp, 31
- comparePagalRezultata
 - Studentas.cpp, 31
- comparePagalVarda
 - Studentas.cpp, 31
- egzaminas
 - Studentas, 9
- funkcions.h, 15
 - FUNKCIONES_H, 16
 - GeneruotiFaila, 16
 - Isvedimas, 16
 - IsvestilFaila, 16
 - LaikoMatavimas, 16
 - Nuskaitymas, 16
 - NuskaitymasIsFailo, 16
 - RandomPazymiai, 16
 - RandomStudentai, 17
 - RankinisIvedimas, 17
 - TestuotiGreiti, 17
- FUNKCIONES_H
 - funkcions.h, 16
- Generatorius.cpp, 18
 - GeneruotiFaila, 18
- GeneruotiFaila
 - funkcions.h, 16
 - Generatorius.cpp, 18
- includes.h, 18
 - INCLUDES_H, 19
- INCLUDES_H
 - includes.h, 19
- input_impl.h, 20
 - Nuskaitymas, 20
- inputbyhand_impl.h, 21
 - RankinisIvedimas, 21
- Isvedimas
 - funkcions.h, 16
 - output_impl.h, 25
- IsvestilFaila
 - funkcions.h, 16
 - output_impl.h, 25
- LaikoMatavimas
 - funkcions.h, 16
 - LaikoMatavimas.cpp, 23
- LaikoMatavimas.cpp, 23
 - LaikoMatavimas, 23
- main
 - v2.0.cpp, 34
- mediana
 - Studentas, 9
- Nuskaitymas
 - funkcions.h, 16
 - input_impl.h, 20
- nuskaitymas_impl.h, 23
 - NuskaitymasIsFailo, 23
- NuskaitymasIsFailo
 - funkcions.h, 16
 - nuskaitymas_impl.h, 23
- operator<<
 - Studentas, 11
 - Studentas.cpp, 31
- operator>>
 - Studentas, 11
 - Studentas.cpp, 31
- operator=
 - Studentas, 9
- output_impl.h, 24
 - Isvedimas, 25
 - IsvestilFaila, 25
- pavarde
 - Zmogus, 12
- pavarde_
 - Zmogus, 13
- paz
 - Studentas, 9
- pazymiairandom_impl.h, 26
 - RandomPazymiai, 26
- RandomPazymiai
 - funkcions.h, 16
 - pazymiairandom_impl.h, 26
- RandomStudentai
 - funkcions.h, 17
 - studentairandom_impl.h, 29
- RankinisIvedimas

- funkcions.h, 17
 - inputbyhand_impl.h, 21
- readStudent
 - Studentas, 10
- rezultatas
 - Studentas, 10
- setEgzaminas
 - Studentas, 10
- setMediana
 - Studentas, 10
- setPavarde
 - Zmogus, 12
- setPaz
 - Studentas, 10
- setRezultatas
 - Studentas, 10
- setVardas
 - Zmogus, 12
- skaiciuotiRezultata
 - Studentas, 10
 - Zmogus, 12
- Strategija1
 - Strategijos.h, 28
- Strategija2
 - Strategijos.h, 28
- Strategija3
 - Strategijos.h, 28
- Strategijos.h, 27
 - Strategija1, 28
 - Strategija2, 28
 - Strategija3, 28
 - STRATEGIJOS_H, 28
- STRATEGIJOS_H
 - Strategijos.h, 28
- StudContainer
 - v2.0.cpp, 34
- studentairandom_impl.h, 29
 - RandomStudentai, 29
- Studentas, 7
 - ~Studentas, 8
 - addPaz, 9
 - egzaminas, 9
 - mediana, 9
 - operator<<, 11
 - operator>>, 11
 - operator=, 9
 - paz, 9
 - readStudent, 10
 - rezultatas, 10
 - setEgzaminas, 10
 - setMediana, 10
 - setPaz, 10
 - setRezultatas, 10
 - skaiciuotiRezultata, 10
 - Studentas, 8
- Studentas.cpp, 30
 - comparePagalPavarde, 31
 - comparePagalRezultata, 31
 - comparePagalVarda, 31
 - operator<<, 31
 - operator>>, 31
- Studentas.h, 31
- testuoti_impl.h, 32
 - TestuotiGreiti, 33
- TestuotiGreiti
 - funkcions.h, 17
 - testuoti_impl.h, 33
- v2.0.cpp, 34
 - main, 34
 - StudContainer, 34
- vardas
 - Zmogus, 13
- vardas_
 - Zmogus, 13
- Zmogus, 11
 - ~Zmogus, 12
 - pavarde, 12
 - pavarde_, 13
 - setPavarde, 12
 - setVardas, 12
 - skaiciuotiRezultata, 12
 - vardas, 13
 - vardas_, 13
 - Zmogus, 12
- Zmogus.h, 34